

bthome-writable — one page for the BTHome maintainers

A briefing, not a draft: facts and links to speak from. The submission is the owner's own words — the Open Home Foundation [AI policy](#) closes contributions believed to be agent-written (D-091).

The ask

Reserve one object ID: 0xFF . Everything else reuses BTHome's own object table, encodings and AES-CCM. And a question rather than a request: the specification is BTHome's to take if the maintainers want it.

Why it is worth having

Gordon Williams (Espruino), its co-designer:

BTHome is a really great tool for sensors, but it's really handy to have a way of controlling bluetooth devices in a low-power, secure way, and nothing in Home Assistant really does that at the moment. It adds that functionality for those that want it but in a way that doesn't complicate BTHome.

His list: clocks and alarms, lights, thermostat targets, movement thresholds, plant waterers, epaper displays.

What it does

A device lists, in its ordinary BTHome advertising, the object types it accepts writes for — object 0xFF , **last in the service data**, so existing receivers skip it. A receiver connects briefly and writes **one BTHome object** to that entry's own GATT characteristic. The write response is the acknowledgement; a writable value is never advertised.

```
advertising  40 00 09 01 61 FF 01 1E      battery, and "entry 1 accepts 0x1E"
write        1E 01  -> 2FAA0001          one object, one characteristic
```

It answers requests already made here

- [bthome-ble #257](#) — "BTHome should define an Object Id for supported events": our declaration, asked for independently in 2025 for a narrower case.
- [#146](#) two-way communication — Ernst79: "we will welcome contributions from others if they want to add this somehow." And [#287](#), controlling Shelly BLU devices.

Re-checked 2026-10-07: no downlink proposed anywhere, 0xFF unassigned.

Evidence

Three benches, two of them other people's. Gordon Williams on a Puck.js: "works great", connections brief, no missed writes. @enaon on an RPi4b with OpenWrt, twenty encrypted lights, six ESPHome proxies and local BlueZ: working over both paths. **Both outside reports found faults this bench could not produce**, each fixed with a test that fails without the fix.

Measured through Home Assistant	
Click to action, 1 s advertising interval	1.7 s , of which 36 ms is the write
Repeated commands	0.31 s
Delivered, two devices	100 % , no retries
Declaration on the air, three writable lights	5 bytes

Two reference implementations, a shared test-vector contract with on-device AES-CCM vectors, 689 automated tests across three suites.

Objections, and the short answers

- Slow, and costly in battery.** 1.7 s click to action, and the idle advertising interval is the device's own choice, unchanged by this.
- Unknown objects break existing receivers.** bthome-ble skips an unknown ID and stops parsing, so the declaration goes last and existing installs lose nothing. Verified against the real parser in CI.
- Writes are a security hole.** BTHome's own AES-CCM both directions, the direction bound into the nonce so no recording replays as another, and a monotonic counter. Without the key there is no write.

Links

<https://github.com/verpj/bthome-writable> — specification spec/PROTOCOL.md , every resolved question with the measurement that resolved it spec/decisions.md , ten-minute replication docs/try-it.md . Discussion: [espruino#8024](#).